

1. INSTALLATION & IMPORTS

Install via pip directly from GitHub:

```
pip install git+https://github.com/ezraair555/py-flexplot.git
```

```
from pyflexplot import (
    flexplot, visualize, compare_fits, added_plot
)
```

2. FORMULA GRAMMAR ANATOMY

Define axes, aesthetics, and panel facets via a single formula string:

```
outcome ~ predictor | color + panel_col | panel_row
```

Element	Role	Mapping
outcome	Dependent / response variable	Y-axis
predictor	Primary independent variable	X-axis
color	Grouping aesthetic variable	Color / Legend
panel_col	1st conditional facet variable	Column panels
panel_row	2nd conditional facet variable	Row panels

3. MAIN SIGNATURE & OBJECT HANDLING

```
flexplot(formula, data, method="auto", **kwargs)
```

Returns a standard **plotnine.ggplot** object. Extend with plotnine layers or draw directly:

```
import pandas as pd
from plotnine import theme_minimal, labs

df = pd.read_csv("data.csv")
p = flexplot("cty ~ hwy | drv", data=df)
p = p + theme_minimal() + labs(title="Mileage Analysis")
p.draw()
```

4. AUTOMATIC PLOT SELECTION

flexplot() detects variable types to select the optimal geom:

Y Variable	X Variable	Rendered Geom
Numeric	Numeric	Scatter + Fitted Curve/Line
Numeric	Categorical	Jittered points + Mean/SE bar
Binary (0/1)	Numeric	Logistic Sigmoid Curve
Categorical	Categorical	Binned Frequency / Jitter grid
Numeric	None (~ 1)	Distribution Histogram + KDE

5. SMOOTHING & FIT METHODS

method=	Algorithm & Description
"auto"	Default. Loess for N < 1000, GAM/Linear for large N.
"lm"	Linear regression line with 95% confidence ribbon.
"loess"	Local polynomial regression (non-parametric smooth).
"logistic"	Binomial generalized linear model (probability response).
"polynomial"	Polynomial regression curve (requires degree arg).
# Linear regression fit line flexplot("income ~ education", data=df, method="lm").draw()	

6. UNIVARIATE DISTRIBUTIONS

Plot single variable density/histograms using intercept-only formula:

```
# Intercept-only creates a distribution histogram
flexplot("income ~ 1", data=df).draw()
```

7. FACETING & MULTI-PANEL GRIDS

Split datasets into subplots cleanly using formula conditioning pipes:

```
# 1D Column Facet
flexplot("sales ~ spend | ad_type", data=df).draw()

# 2D Grid: Columns by region, rows by quarter
flexplot("sales ~ spend | region | quarter",
data=df).draw()

# Color aesthetic + 1D Facet
flexplot("sales ~ spend | channel + region",
data=df).draw()
```

8. COMMON FLEXPLOT ARGUMENTS

Parameter	Type	Purpose
alpha	float (0-1)	Transparency of points to handle overlap
jitter	tuple/float	Manual jitter noise for discrete points
se	bool	Display confidence interval bands (Default: True)
bins	int	Number of histogram bins for 1D distributions
ghost	bool	Show background reference sample across facets

9. QUICK PLOT TEMPLATES

```
# Categorical vs Numeric with group coloring
flexplot("satisfaction ~ dept | gender", data=df).draw()

# Binary response with logistic fit
flexplot("churn ~ tenure", data=df,
method="logistic").draw()
```

1. VISUALIZE() FITTED MODELS

Generates diagnostic actual vs. predicted or prediction sweep plots:

```
import statsmodels.formula.api as smf

mod = smf.ols("performance ~ tenure + training",
data=df).fit()
visualize(mod).draw()

# Predict on new/held-out test data
visualize(mod, data=test_df).draw()
```

2. COMPARE_FITS() MULTI-MODEL

Overlays fits of competing models against the raw scatter:

```
m1 = smf.ols("y ~ x", data=df).fit()
m2 = smf.ols("y ~ x + I(x**2)", data=df).fit()

compare_fits(
    "y ~ x", data=df,
    model1=m1, model2=m2,
    labels=["Linear", "Quadratic"]
).draw()
```

3. ADDED_PLOT() PARTIAL REGRESSION

Isolates partial effect of X on Y after residualizing other variables:

```
added_plot("salary ~ experience + age + education",
data=df).draw()
```

4. DEEP LEARNING: NEURALNETFIT

Wraps PyTorch / Keras models into standard visualize() pipelines:

```
from pyflexplot.flex_nn import NeuralNetFit,
permutation_importance

fit = NeuralNetFit(
    model=torch_model,
    response_var="target",
    predictor_names=["feat1", "feat2", "feat3"],
    backend="torch"
)
visualize(fit, data=df).draw()
```

5. PERMUTATION FEATURE IMPORTANCE

```
imp = permutation_importance(
    fit, X=df[["x1", "x2"]], y=df["y"],
    metric="mae", n_repeats=10, seed=42
)
print(imp.sort_values("importance", ascending=False))
```

6. MIXED_MODEL() DATA GENERATOR

Simulates complex hierarchical / multilevel datasets:

```
from pyflexplot.bluepill import mixed_model, estimate_sd

synth_df = mixed_model(
    fixed=[10.0, 0.2, 0.5],
    random=[0.1, 0.1],
    sigma=0.3,
    clusters=15,
    n_per=[10, 30],
    seed=42
)
```

7. BIOSTATS UTILITIES (FIFER)

```
from pyflexplot.stats import (
    model_comparison, estimates,
    p_format, eliminated_columns, color_table
)

# Delta AIC/BIC table
model_comparison(m1, m2)

# Effect sizes and parameter intervals
estimates(mod)

# Formatted p-values
p_format(0.00034, digits=3) # Returns "< .001"

# Colored correlation matrices
color_table(df.corr(), cmap="coolwarm")
```

8. STRUCTURAL EQUATION MODELS (SEMOPY)

```
from pyflexplot.sem import (
    hopper_plot, disturbance_plot, measurement_plot
)

hopper_plot(sem_mod).draw()
disturbance_plot(sem_mod, "F1", "F2", data=df).draw()
measurement_plot(sem_mod, "F1", "indicator1",
data=df).draw()
```

9. EMPIRICAL BAYES (EBBR)

```
from pyflexplot.ebbr import fit_beta_prior,
add_ebb_estimate

prior = fit_beta_prior(df["successes"], df["trials"])
df_shrunk = add_ebb_estimate(
    df, "successes", "trials", prior=prior
)
```